Section 3: System Design Fundamentals

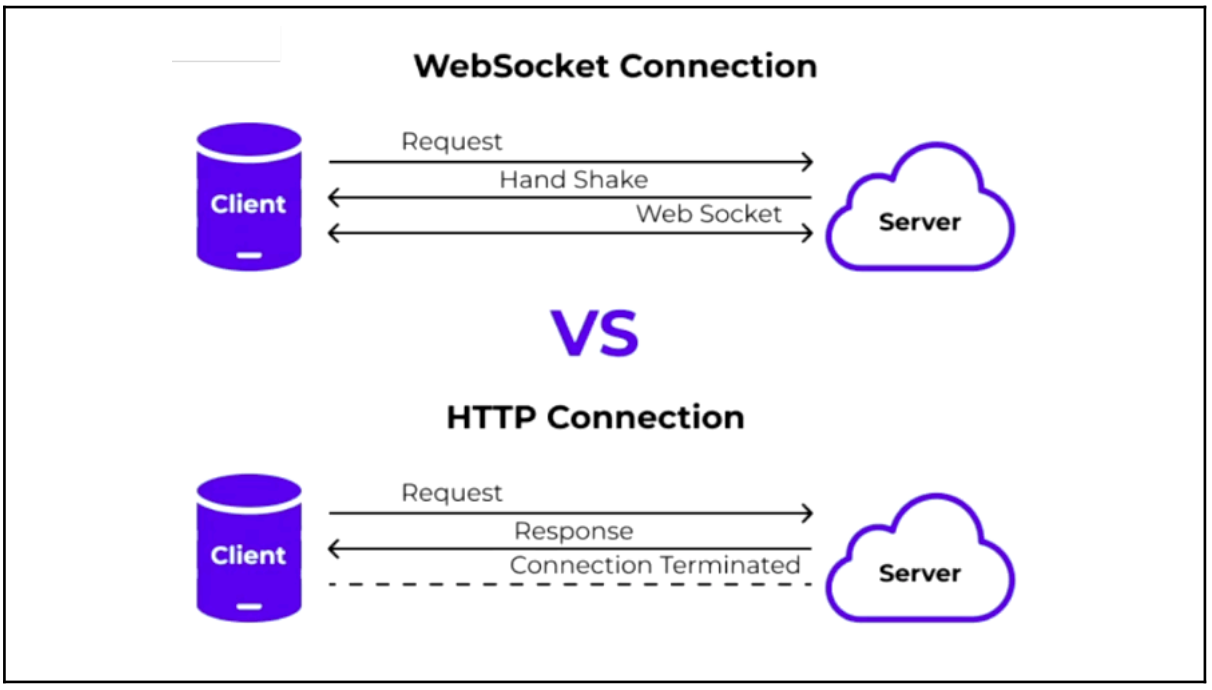
Real-Time Communication Protocols

Real-time communication protocols are a very essential topic in system design. In modern applications, delivering instant updates to users is critical, whether it's a live chat application, stock price tracker or real-time notification system. The right communication protocol can make all the difference here.

In this session, we will explore the major approaches to real-time communication. We will look at web-sockets and long polling. We will dive into how they work, their advantages, when to use each one and some-real world examples where they applied. By the end of this session you will have a solid understanding of these protocols and be able to choose the right one when designing scalable and efficient systems.

Introduction to Real-Time Communication

In today's world users expect instant updates, whether it's receiving a chat message, tracking a stock price or playing an online game. This is where real-time communication comes in. Real-Time Communication refers to the continuous exchange of data with minimal latency. So unlike traditional systems where updates happen only when a user manually requests them, real-time systems push updates automatically and instantly. Think about your whatsapp, when you send a message, it appears instantly for the receiver or stock market app where the prices get updated in real-time.



Why is real-time communication important? Imagine this whatsapp chat application where you had to refresh the page to see any new messages. This would frustrate the user. So real-time communication is critical for providing a smooth user experience in applications like instant messaging, live stock market updates, multiplayer gaming, live sports and score streaming. Without real-time communication, these experiences would feel slow, outdated and unresponsive.

So why cannot HTTP solve this problem? The traditional web follows a request response model where the client requests the data and the server responds. While this works for static content, it has the major limitation for real-time application. The below are some limitations:

- **High Latency:** The client has to wait for the updates.
- **Inefficiency:** The server only responds when asked, causing unnecessary delays for clients.
- **Scalability:** We can use HTTP to continuously poll for updates, but it will not be a scalable solution. It will increase the server load over time.

So to solve these issues, various techniques have been developed to improve real-time communications. So there are multiple ways to achieve real-time updates and each of them has its trade-offs. The below are some of the ways to achieve real-time communication:

➔ **Polling:** The client will repeatedly ask the server for updates at fixed intervals.

➔ **WebSockets:** Websocket is a persistent connection. A full duplex connection that allows instant two-way communication between client and servers.

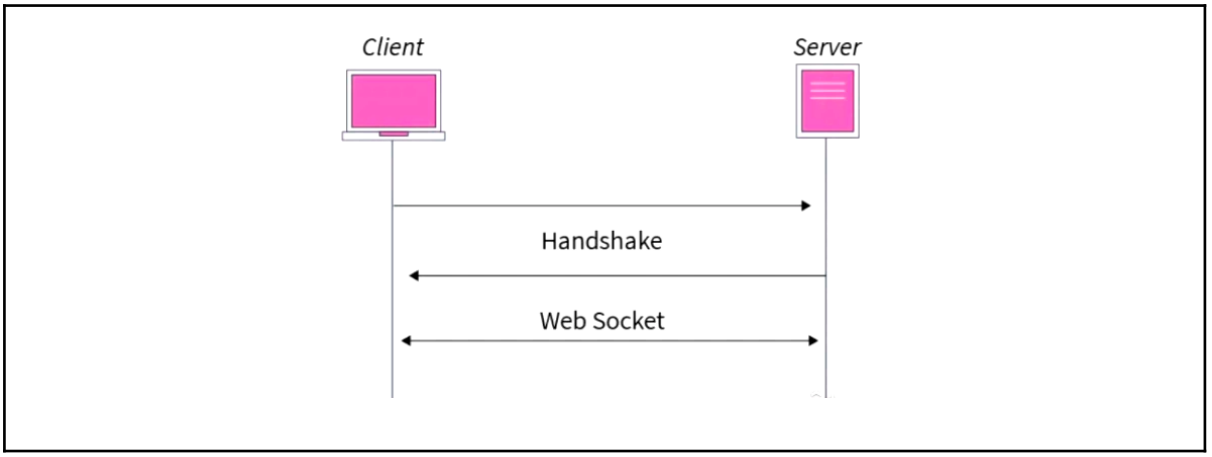
➔ **Server-Sent Events (SSE):** It's an unidirectional stream where the server pushes updates to the client, unlike HTTP. And then we have long polling.

➔ **Long Polling:** It's a variation of polling where the server holds the request. Rather than responding to every request, the server will hold the request open until there is new data available.

Each of these techniques has its advantages and is suited for different use cases. In this session, we will explore WebSockets and Long Polling in depth to understand how they enable efficient real-time communications.

WebSockets: Persistent Full-Duplex Communication

In modern applications, real-time data exchange is crucial and one of the most efficient ways of achieving it is through WebSockets. WebSockets provide a persistent full duplex connection between the client and server over a TCP connection., and thus enabling a real-time bidirectional communication. Unlike traditional HTTP, where a client requests data and waits for the response, WebSocket allows continuous communication without needing to establish a new connection for every message. So the connection is persistent and stays open once it is established. Also, it is full duplex meaning both the client and server can send and receive messages at any time. It eliminates the overhead of repeating HTTP requests making it much more efficient. Think of it like a direct phone call rather than sending the separate letters back and forth. Once connected, both sides can talk freely until one decides to hang up.



The WebSockets actually have a step-by-step process how they communicate:

- **Step1:** Client requests to use the WebSocket. So initially, the communication starts with the regular HTTP request, but the client includes a special header requesting to switch to a WebSocket. This tells the server to upgrade our communication to a WebSocket connection.
- **Step2**: The server accepts this request and keeps the connection open. But the server has to support WebSocket. So if the server supports WebSockets, It responds to a 101 written code and switches the protocol to WebSocket. At this point, the connection is established and it will remain open until either the client or the server decides to close it.
- **Step3:** The data is exchanged in real-time using the frames. So once the connection gets open, messages can be sent in either direction at any time, unlike traditional HTTP, where each request requires a new connection.
- **Step4:** When the communication is no longer needed, either the client or the server can close the connection using a special WebSocket close frame.

So to summarize, WebSockets offer a powerful way to enable real-time communication by maintaining a persistent bidirectional connection. They are widely used in applications like chat systems, live notifications, stock market updates and gaming.

Advantages and Use Cases of Web Sockets

WebSockets are very powerful and commonly used for real-time communication due to the following advantages:

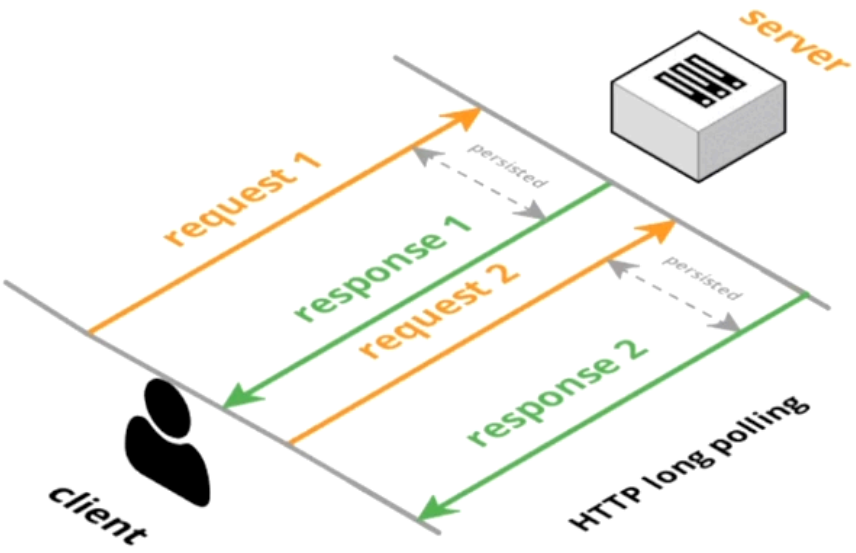
- ➔ **Persistent Connection:** Unlike single HTTP requests, WebSockets maintain a single open connection, eliminating the need to repeatedly establish new connections. This significantly reduces latency and makes the interactions feel much more instant.
- ➔ **Reduces Overhead:** It reduces overhead compared to HTTP polling. With polling, the client must constantly send requests even if there is no new data. WebSockets eliminates this unnecessary traffic and thus reduces bandwidth usage and server load.
- ➔ **Efficient Real-Time Communication:** It is very efficient for real-time applications. Since WebSockets enable bidirectional event driven communication, they are prepared for applications that require instant update without delays.

Some of the use cases of WebSockets are:

- **Live Chat Applications**: WhatsApp, Slack, Discord
- **Stock Market Price Updates:** NASDAQ, Robinhood
- **Multiplayer Online Games**: Fortnite, Call of Duty
- **Collaborative Tools:** Google Docs, Figma

So to summarize, WebSockets provide a low latency, high efficiency solution for real-time communication. Whether it’s a chat application, stock price update application, gaming application, WebSockets ensure that that data flows instantly and efficiently.

Long Polling: Simulating Real-Time with HTTP



Long polling is a technique that allows us to achieve real-time communication even when WebSockets are not available. In this technique, the client sends an HTTP request to the server but instead of the server responding immediately, the server will hold the request open until the new data is available. It simulates real-time updates using HTTP only and it is much more efficient than regular polling, because in regular polling the constant request will be sent to the server whereas in this the single request will be held by the server. This is very useful when WebSockets are not supported or are not needed.

In regular polling the client keeps on sending requests at fixed intervals whether or not new data is available. This waste resources and creates unnecessary server load. With long pooling the client sends a request but the server waits until there is a new data available before responding. This reduces unnecessary requests and provides faster updates whenever there is any change on the server side.

The below are the step-by-step working mechanism of long pooling:

- **Step1:** The client makes an HTTP request to the server asking for new data.
- **Step2:** Instead of responding immediately the server will keep the request open until new data is available.
- **Step3:** Once the server has the new data it will send the response to the client.
- **Step4:** The client will get the response and it will immediately send another request to wait for the next update. It will close the first one before sending another request.

This cycle will repeat keeping the data fresh while reducing unnecessary traffic. So to summarize, long polling provides a more efficient way to achieve real-time updates over HTTP. Not as good at WebSockets but still better than traditional HTTP. It’s not as fast as a WebSocket but it is useful in environments where WebSockets aren’t supported or even simpler solutions are needed.

When to Use WebSockets vs. Long Pulling

- Use WebSockets when:

If high-frequency bi-directional data exchange is needed, if our application requires continuous instant communication between clients and server, WebSockets are the best choice. This is especially true when both parties need to send messages at any time. Whenever low latency is critical, in use cases like real-time chat, stock trading, multiplayer gaming, where every millisecond counts. WebSockets provide a persistent low latency connection ensuring data is transmitted without delays.

- Use Long Pooling when:

If WebSockets are not supported or if they are an overkill, then we switch to long pooling. Long pooling provides a very good alternative to WebSockets and using standard HTTP protocols. You can also use long pooling when periodic updates are sufficient. If our updates don't need to be sent every second but still need to appear quickly, long pooling works well. It allows the server to push updates when they are available rather than the client constantly checking for it.

- Real World Examples:
    - **WebSockets is used by:** Slack, Stock Exchange.
    - **Long Pooling is used by:** Twitter, IoT devices.

Choosing the right approach depends on your use case. WebSockets are ideal for instant high-frequency communication while long pooling works well for periodic updates when WebSockets are not an option.

Summary & Final Takeaways

- WebSockets = Persistent, full-duplex communication.
- Long Polling = Simulated real-time via HTTP requests.
- Choose based on latency needs and infrastructure.
- What's next:
    - Modern API Protocols - Beyond REST(gRPC, GraphQL)

Interview Questions

- Basic Questions
  - What is real-time communication, and why is it important?
  - How does WebSockets work, and how does it differ from traditional HTTP?
  - Explain the WebSocket handshake process.
  - What is long polling, and how does it work?
- Intermediate Questions
  - What are the advantages of WebSockets over long polling?
  - In what scenarios would you prefer long polling over WebSockets?
  - How does WebSockets handle connection failures or network interruptions?
  - Can you use WebSockets with load balancers? If yes, how?
  - What are some challenges of scaling WebSockets in a distributed system?